

Software Quality Management Framework

Doc Ref: ACQO-DOC-007 | v1.0 | 2025

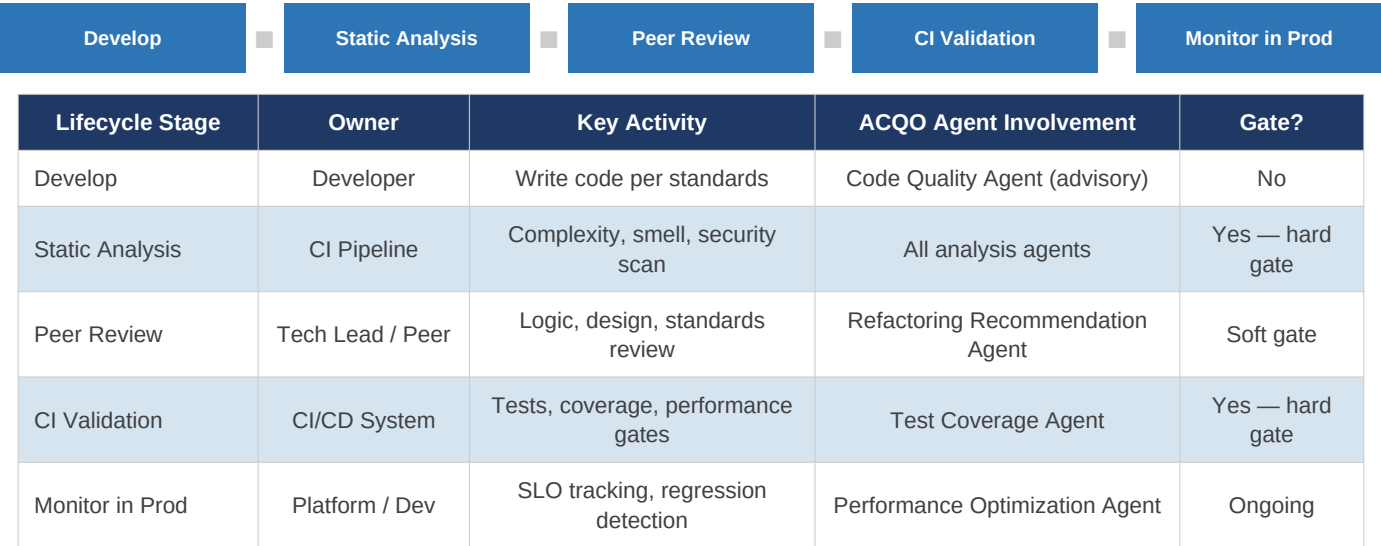
Autonomous Code Quality & Optimization Intelligence System

1. Framework Introduction

The Software Quality Management Framework (SQMF) defines the lifecycle, governance, and continuous improvement practices that guide how the ACQO system evaluates, scores, and prioritises code quality actions across all monitored repositories. This excerpt covers the code quality lifecycle, review practices, technical debt management, and agent-based routing logic.

2. Code Quality Lifecycle

Quality is managed across five continuous lifecycle stages for every code change:



3. Code Review Best Practices

- **Scope:** Pull requests should be limited to a single logical change. PRs exceeding 400 lines of diff require explicit justification and are auto-flagged by ACQO.
- **Review Checklist:** Reviewers must verify correctness, security compliance, test coverage, adherence to SOLID principles, and absence of code smells.
- **Self-Review First:** Authors must self-review their diff before requesting a peer review to reduce trivial back-and-forth cycles.
- **Constructive Feedback:** Review comments must be specific, actionable, and reference the relevant guideline (e.g. "Cyclomatic complexity exceeds threshold — see ACQO-DOC-004 §3").
- **Approval Requirement:** At least one Tech Lead approval required for changes to security-critical modules (user-auth-service, payment-service).
- **Merge Window:** Changes to production-critical services must not be merged in the 2-hour pre-deployment freeze window.

4. ACQO Query Classification & Agent Routing

The ACQO system routes developer queries to specialized agents based on query type, risk level, and expected retrieval mode:

Query Type	Example	Retrieval Mode	Agent Routed To	Escalation?
Code Quality Guidance	How to reduce cyclomatic complexity?	RAG	Code Quality Agent	No
Metrics Lookup	Files with complexity above 18	SQL	Code Quality Agent	No
Hybrid Optimization	Which high-complexity files to refactor first?	RAG + SQL	Refactoring Agent + QA Agent	Conditional
High-Risk Change	Deploy module with security vulnerabilities?	Multi-Agent	All Agents	Yes
Escalation Scenario	Override security rule to meet deadline	Human Only	Human Developer / Tech Lead	Always

5. Technical Debt Management Strategies

Technical debt accumulates when short-term shortcuts defer necessary quality work. The ACQO framework classifies and manages debt across four categories:

Debt Type	Definition	Current Example	Remediation Priority
Code Debt	Violations of coding standards and complexity limits	AuthManager.java — 7 smells, CC=19.1	High — next sprint
Test Debt	Insufficient test coverage in critical modules	auth-service coverage 74.4% vs 90% SLO	High — block features
Architecture Debt	Design pattern violations or missing abstractions	Layering violations in payment-service	Medium — scheduled sprint
Documentation Debt	Missing or outdated technical documentation	data_pipeline.py — no docstrings on 8 funcs	Low — rolling basis

6. Continuous Improvement Practices

- **Quality Retrospectives:** Every sprint retrospective must include a 15-minute quality review covering ACQO agent findings from the sprint.
- **Debt Ratio Tracking:** Track the ratio of technical debt remediation work to feature work. Target: ≤ 20% of sprint capacity on debt.
- **Quality Scorecards:** Monthly scorecards are generated per repository summarising complexity trends, coverage trajectory, and smell resolution rate.
- **Agent Feedback Loop:** Developer feedback on ACQO agent recommendations is captured and used to improve routing accuracy and recommendation quality.
- **Refactoring Sprints:** Dedicated refactoring sprints are scheduled quarterly for any repository with maintainability index below 70.

7. Roles & Responsibilities

Role	Primary Responsibility	ACQO Interaction	Approval Authority
Developer	Write quality code, self-review, fix ACQO flags	Query interface, view recommendations	Own PR changes
Tech Lead	Code review, architecture oversight, debt planning	Review escalated agent findings	Approve security-critical PRs
QA Engineer	Test strategy, coverage monitoring, regression	Test Coverage Agent outputs	Test gate sign-off
Platform/DevOps	CI/CD pipeline, monitoring, SLO management	Performance Optimization Agent integration	Pipeline configuration
ACQO Agent System	Analysis, recommendations, escalation routing	Advisory only — no auto-deploy or auto-merge	None — advisory role

Human Escalation Decision Flow



■ Framework Note: ACQO agents provide advisory recommendations only. All deployment decisions, security overrides, and architecture changes require human approval. Agents never auto-approve Tier 3+ risk actions.

✓ Recommended Practices	✗ Prohibited Practices
✓ Treat ACQO agent findings as first-class inputs to sprint planning	✗ Allow ACQO agents to make deployment decisions autonomously
✓ Resolve security vulnerability flags before any feature work continues	✗ Defer technical debt remediation indefinitely in favour of feature velocity
✓ Document every override of a CI quality gate with justification and owner	✗ Ignore repeated ACQO recommendations as "noise" without investigating root cause